Joint Intent Classification and Slot Filling on ATIS: from Recurrent Networks to BERT

Andrea Giampietro

University of Trento

Abstract

We train a single model to recognise the intent of a flight-domain request and to label its words with semantic slots. On ATIS, a bidirectional LSTM with dropout raises the slot F1 of an LSTM baseline from 92.24 to 94.59 and its intent accuracy from 93.44% to 94.94%. Fine-tuning BERT, with the slot labels aligned to the first WordPiece of every word, reaches 95.49 slot F1 and 97.54% intent accuracy (means over 5 runs).

1. Introduction

Spoken language understanding maps a request to a semantic frame: the intent says what the user wants and the slots carry its arguments. In *list flights from la guardia to burbank* the intent is `flight`, *la guardia* fills `fromloc.airport_name` and *burbank* fills `toloc.city_name`. The two tasks are related and are commonly learnt by one model with two outputs [1]. We study them on ATIS [2]: part A extends an LSTM baseline with a bidirectional encoder [3] and dropout [4], and part B fine-tunes BERT [5] on both tasks at once [6]. The code is available at `https://github.com/giampyhatesyou/Natural_Language_Processing`.

2. Implementation details

Data and evaluation. We use a preprocessed version of ATIS with 4,978 training and 893 test utterances, 26 intents and 129 slot labels in IOB format [7]. 10% of the training set, stratified by intent, is held out as dev set. Intents are evaluated by accuracy and slots by chunk-level F1 with the conlleval script [8].

Part A. Words are embedded in 300 dimensions and encoded by an LSTM with 200 units per direction, which skips the padding through packed sequences. A linear slot layer reads every encoder state and a linear intent layer the final state; in the bidirectional model the intent layer receives the last forward state concatenated with the backward state at the first word, which together summarise the whole utterance. Dropout ($p = 0.3$) is applied to the embeddings and before both output layers. The loss is the sum of the two cross-entropies, minimised with Adam [9] (lr 10^{-3}, batch size 128, gradients clipped to norm 5).

Part B. `bert-base-uncased` feeds a linear intent layer with the pooled `[CLS]` representation and a linear slot layer with the last-layer representation of every sub-token (dropout 0.1). WordPiece [10] splits 2.9% of the test words, but 6.1% of the words inside a slot (city and airport names, aircraft and fare codes), and one test utterance in four contains a split word. Since ATIS labels whole words, the label of a word is assigned to its first sub-token, while the other sub-tokens, `[CLS]`, `[SEP]` and the padding get the ignore index of the loss (Table 2). At inference the prediction of a word is read from its first sub-token, so the slot F1 is computed on the original words with the same script as part A. The pre-trained encoder is loaded once per run and only the two output layers start from scratch. We fine-tune with AdamW [11] (lr $5 \cdot 10^{-5}$, weight decay 0.01 except on biases and LayerNorm), a linear warm-up over the first 10% of the steps followed by a linear decay, batch size 32 and at most 10 epochs.

Table 1: *Test results on ATIS: mean ± sample standard deviation over 5 runs, each evaluated at its best dev epoch. *: significantly better than the previous row (Welch's t-test, $p < 0.05$).*

Model	Slot F1	Intent acc. (%)
LSTM	92.24 ± 0.54	93.44 ± 0.47
BiLSTM	$94.13 \pm 0.45^{*}$	$94.54 \pm 0.15^{*}$
BiLSTM + dropout	94.59 ± 0.30	94.94 ± 0.35
BERT	$\mathbf{95.49} \pm 0.11^{*}$	$\mathbf{97.54} \pm 0.24^{*}$

Model selection. Each configuration is trained 5 times with different seeds. In every run the dev slot F1 is computed after each epoch, training stops after 10 (part A) or 3 (part B) epochs without improvement, and the best weights are restored before testing. We report mean and sample standard deviation over the runs and compare consecutive models with Welch's t-test ($\alpha = 0.05$).

3. Results

Table 1 reports mean and sample standard deviation of the test scores over the five runs. The bidirectional encoder gives the largest gain on slot filling (+1.89 F1, $p < 0.001$), since the backward direction lets the label of a word depend on the words that follow it. Dropout adds +0.46 F1 and +0.40 points of accuracy, but with five runs neither gain is significant ($p = 0.10$ and $p = 0.06$): they are within the variation between seeds. BERT is the best model on both tasks ($p \leq 0.001$): the margin is large on intents (+2.60 points), where a pre-trained representation of the whole sentence helps most, and smaller on slots (+0.90 F1), which in ATIS largely depend on local lexical cues already captured by the BiLSTM; it also has the smallest spread on slots (± 0.11 F1). Averaging over runs matters: the BERT run with the best dev F1, the one kept as checkpoint, has the lowest test F1 of the five (95.33). Thanks to the first-sub-token alignment, the WordPiece segmentation does not penalise BERT: all the models are scored on exactly the same words.

4. References

[1] B. Liu and I. Lane, "Attention-based recurrent neural network models for joint intent detection and slot filling," in *Proc. Interspeech*, 2016, pp. 685–689.

[2] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington, "The ATIS

Table 2: *Training targets of BERT for the test utterance* list flights from la guardia to burbank*: only the first sub-token of each word has a label.*

Sub-token	Word	Target
[CLS]	–	ignored
list	list	O
flights	flights	O
from	from	O
la	la	B-fromloc.airport_name
guard	guardia	I-fromloc.airport_name
##ia	guardia	ignored
to	to	O
bu	burbank	B-toloc.city_name
##rba	burbank	ignored
##nk	burbank	ignored
[SEP]	–	ignored

spoken language systems pilot corpus," in *Proc. Speech and Natural Language Workshop*, 1990.

[3] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186.

[6] Q. Chen, Z. Zhuo, and W. Wang, "BERT for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.

[7] L. A. Ramshaw and M. P. Marcus, "Text chunking using transformation-based learning," in *Proc. Third Workshop on Very Large Corpora*, 1995.

[8] E. F. Tjong Kim Sang and S. Buchholz, "Introduction to the CoNLL-2000 shared task: Chunking," in *Proc. Conference on Computational Natural Language Learning (CoNLL)*, 2000, pp. 127–132.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[10] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.

[11] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.